Licensing



© Copyright 2004-2026, Bootlin
Creative Commons BY-SA 3.0
Corrections, suggestions, contributions and translations are welcome!



Managing licenses



Tracking license changes

- The license of an external project may change at some point.
- The LIC_FILES_CHKSUM tracks changes in the license files.
- If the license's checksum changes, the build will fail.
  - The recipe needs to be updated.

```
LIC_FILES_CHKSUM = "                                    \
    file://COPYING;md5=...                              \
    file://src/file.c;beginline=3;endline=21;md5=..."
```

- LIC_FILES_CHKSUM is mandatory in every recipe, unless LICENSE is set to CLOSED.



Package exclusion

- We may not want some packages due to their licenses.
- To exclude a specific license, use INCOMPATIBLE_LICENSE
- To exclude all GPLv3 packages:

```
INCOMPATIBLE_LICENSE = "GPL-3.0* LGPL-3.0* AGPL-3.0*"
```

- License names are the ones used in the LICENSE variable.



Commercial licenses

- By default the build system does not include commercial components.
- Packages with a commercial component define:

```
LICENSE_FLAGS = "commercial"
```

- To build a package with a commercial component, the package must be in the LICENSE_FLAGS_ACCEPTED variable.
- Example, `gst-plugins-ugly`:

```
LICENSE_FLAGS_ACCEPTED = "commercial_gst-plugins-ugly"
```



Listing licenses

OpenEmbedded will generate a manifest of all the licenses of the software present on the target image in `$BUILDDIR/tmp/deploy/licenses/<image>/license.manifest`

```
PACKAGE NAME: busybox
PACKAGE VERSION: 1.31.1
RECIPE NAME: busybox
LICENSE: GPL-2.0-only & bzip2-1.0.4

PACKAGE NAME: dropbear
PACKAGE VERSION: 2019.78
RECIPE NAME: dropbear
LICENSE: MIT & BSD-3-Clause & BSD-2-Clause & PD
```

You can also include the manifest and individual licenses in the root filesystem:

- Either use `COPY_LIC_DIRS = "1"` and `COPY_LIC_MANIFEST = "1"`
- Or use `LICENSE_CREATE_PACKAGE = "1"` to generate and install packages including the license files.



Providing sources

OpenEmbedded provides the `archiver` class to generate tarballs of the source code, to meet the requirements of some licenses:

- Use `INHERIT += "archiver"`
- Set the ARCHIVER_MODE variable, the default is to provide patched sources. To provide configured sources:

```
ARCHIVER_MODE[src] = "configured"
```

- The `archiver` class stores all source archives, even for software whose license does not require to provide the sources
  - The documentation provides a sample script to filter only GPL licenses



Generating a Software Bill of Materials (SBoM)

Instead of generating license information and source tarballs separately, OpenEmbedded can actually generate an SBoM, describing:

- **Sources** for target and host components
- **Licenses** of such components
- **Dependencies** between such components
- **Applied changes**, in particular fixes for **known vulnerabilities**.

This SBoM is generated in the standard SPDX format, which you can feed to tools supporting SPDX.



Usefulness of SPDX SBoM

SPDX SBoM can be attached to a software delivery, and used for:

- License compliance assessment
- Vulnerability assessment. You can use the SBoM to check whether your software supply chain is impacted by currently known vulnerabilities, both in host and target packages.

The US government is pushing for having such information in all software it procures and will probably make it mandatory soon.

- `sbom-cve-check` tool from Bootlin can be used to track CVEs from SBOMs: https://github.com/bootlin/sbom-cve-check



How to create SPDX3 SBoM with OpenEmbedded

- The JSON SPDX file for the image will be generated in `tmp/deploy/images/MACHINE/`
- You can then set optional variables:
  - SPDX_PRETTY: Make generated files more human readable (newlines, indentation)
  - SPDX_ARCHIVE_PACKAGED: Add compressed archives of the files in generated target packages.
  - SPDX_INCLUDE_SOURCES: Add descriptions of the source files for host tools and target packages.
  - SPDX_ARCHIVE_SOURCES: Add archives of these source files themselves (when SPDX_INCLUDE_SOURCES is set).
- Note: in Yocto Scarthgap, enabling SPDX3 must be done by setting `INHERIT += "create-spdx-3.0"` and `INHERIT:remove = "create-spdx"` in a configuration file



Example IMAGE-MACHINE.spdx.json output


```
{
  {
    "type": "software_Package",
    "spdxId": "http://spdx.org/spdxdocs/attr-native-1cdc91e2-e63e-5c7a-a86c-844bc9b432c9/91fe9184958e628a
e698fd04f536f49ab3e90b5f9cef4affc5306e9c8883/source/1",
    "creationInfo": "_:CreationInfo231",
    "name": "attr-2.5.2.tar.gz",
    "verifiedUsing": [
      {
        "type": "Hash",
        "algorithm": "sha256",
        "hashValue": "39bf67452fa41d0948c2197601053f48b3d78a029389734332a6309a680c6c87"
      }
    ],
    "software_primaryPurpose": "source",
    "software_downloadLocation": "https://download.savannah.gnu.org/releases/attr/attr-2.5.2.tar.gz"
  },
  {
    "type": "software_File",
    "spdxId": "http://spdx.org/spdxdocs/attr-native-1cdc91e2-e63e-5c7a-a86c-844bc9b432c9/91fe9184958e628a
e698fd04f536f49ab3e90b5f9cef4affc5306e9c8883/source/3",
    "creationInfo": "_:CreationInfo231",
    "name": "0001-attr.c-Include-libgen.h-for-posix-version-of-basenam.patch",
    "verifiedUsing": [
      {
        "type": "Hash",
        "algorithm": "sha256",
        "hashValue": "c41c6435e06c69d9c0338cc573d5f69a58878944b05ceaf9d7a8935c06154b87"
      }
    ],
    "software_primaryPurpose": "patch"
  },
}
```



Further resources about SPDX SBoM

- Yocto project documentation:
  https://docs.yoctoproject.org/dev/dev-manual/sbom.html
- Joshua Watt: Automated SBoM generation with OpenEmbedded and the Yocto Project (FOSDEM 2023)
  https://youtu.be/Q5UQUM6zxVU
- SPDX project homepage:
  https://spdx.dev